

Milestone 3: Critical Evaluation and Transfer

Christian Percival

Purpose

- This milestone evaluates the strengths, limitations, risks, assumptions, and applicability of overrun handling.
- The goal is not to say whether the paper is simply good or bad.
- The goal is to understand where the approach works, where it may fail, and how it transfers to realistic embedded systems.

Strengths of the Approach

- Handles variable execution times more efficiently than pure WCET scheduling.
- Allows higher normal-case task frequencies.
- Improves control performance when normal execution time is much smaller than WCET.
- Uses bandwidth reservation to limit interference between tasks.
- Provides temporal isolation.
- Maintains hard real-time guarantees under bounded worst-case assumptions.
- CBS_{hd} improves standard CBS by handling partial overruns more accurately.
- The approach is strongly motivated by realistic control applications such as visual tracking.

Limitations

- Requires a known or bounded WCET.
- WCET estimation is difficult on modern processors.

- Cache, interrupts, operating system behavior, and background processes can affect timing.
- Requires knowledge of normal computation time.
- Normal computation time may change depending on input data.
- Remaining computation time may be difficult to estimate accurately.
- Server-based scheduling introduces runtime overhead.
- Implementation requires real-time scheduling support.
- A simple desktop operating system cannot provide the same guarantees as a real-time kernel.

Risks and Hidden Assumptions

- If WCET is underestimated, hard deadlines may still be missed.
- If normal execution time is underestimated, overruns may occur too often.
- If server budget is too small, overhead may become too high.
- If server budget is too large, overrun handling becomes less precise.
- If a task overruns while holding a shared resource, blocking problems can occur.
- The theory assumes bounded behavior, but real image processing can have input-dependent complexity.

Runtime Implications

- Budget management costs CPU time.
- Deadline postponement costs CPU time.
- Smaller chunks lead to more frequent budget exhaustion.
- More frequent budget exhaustion leads to more scheduler overhead.
- The paper reports that overhead must be considered when choosing server budget.

Memory and Scalability Implications

- Each task needs server state:
 - current budget,
 - server period,
 - current deadline,
 - remaining computation estimate.
- Memory overhead is probably small for a small number of tasks.
- Analysis becomes more complex as the number of tasks increases.
- Resource sharing makes the analysis more complex.
- More tasks mean more possible interactions and more scheduling decisions.

Suitable Application Examples

- Robotic visual tracking:
 - normal case: object found quickly,
 - overrun case: larger search region needed.
- Radar tracking:
 - normal case: target found near predicted position,
 - overrun case: wider search required.
- Sensor fusion:
 - normal case: simple sensor update,
 - overrun case: additional filtering or correction required.
- Autonomous systems:
 - normal case: predictable environment,
 - overrun case: unexpected obstacle or uncertain sensor data.

Unsuitable or Problematic Application Examples

- Systems where WCET cannot be bounded realistically.
- Systems where tasks must not be delayed after an overrun.
- Systems with extremely limited processor power where server overhead is too expensive.
- Systems where object detection uses complex AI models with highly unpredictable execution time.
- Systems where shared resources are held for long unpredictable times.
- Very simple microcontroller applications where fixed WCET scheduling is already sufficient.

Transfer to Student Prototype

- A full real-time kernel implementation is probably too large for this seminar.
- A controlled simulation is more realistic.
- The prototype can demonstrate:
 - normal execution time,
 - occasional overruns,
 - WCET-based inefficiency,
 - deadline misses without overrun handling,
 - improvement using simplified budget/deadline handling.
- Python and OpenCV can be used for the demonstration.
- Synthetic images can be generated to control easy, medium, hard, and worst-case inputs.

Possible Extensions

- Implement a simple image-processing overrun demo.
- Compare WCET-based scheduling, normal-budget scheduling, and simplified overrun handling.
- Add plots:
 - execution time per frame,

- deadline misses per strategy,
 - CPU utilization per strategy,
 - average frequency per strategy.
- Later extension:
 - run the same experiment on Raspberry Pi,
 - compare desktop timing and embedded Linux timing,
 - measure timing jitter.

Critical Conclusion

- The approach is strong because it addresses a real weakness of WCET-only scheduling.
- It is especially useful when normal execution time is much smaller than WCET.
- It is less useful when WCET is close to normal execution time.
- The main practical challenge is accurate timing estimation and implementation overhead.
- For the seminar, a simplified prototype is suitable as a demonstration, but it should not be presented as a full real-time scheduler.